

Lab 3 中断处理实验报告

[代码仓库地址](#)

1. 实验目的

- 理解 RISC-V 多核 CPU 启动流程及 hart 概念。
- 掌握 S-mode trap 的配置与中断处理流程。
- 实现基于 timer interrupt 的多核时钟管理系统。
- 实现 UART 中断处理，实现键盘输入响应。
- 通过 QEMU 模拟验证中断系统和 timer tick 功能。

2. 实验原理

2.1 多核 hart

- 每个 hart 独立运行，hart0 负责全局初始化（trap vector、timer 等），其他 hart 等待启动信号。

2.2 Sstc（Supervisor Software Timer Compare）中断处理原理

Sstc（**Supervisor Software Timer Compare Extension**）是 RISC-V 在新版特权级规范中引入的一种简化 **S 模式** 定时中断机制。它允许操作系统在 **S 模式** 下直接管理时钟中断，而无需依赖 M 模式固件（如 OpenSBI）转发，从而降低延迟、简化软件栈。

基本原理

在传统 CLINT 方案中：

- 定时器寄存器位于 M 模式地址空间（mtime 与 mtimecmp），
- M 模式定时器中断发生后，由 M 模式 trap handler 处理，再通过 **软件委托**（mip.stip）发往 S 模式。

而 **Sstc 扩展** 允许在 **S 模式** 直接设置定时比较寄存器：

- stimecmp：S 模式可写的定时比较寄存器；
- stime：S 模式可读的当前时间计数；
- 当 `stime >= stimecmp` 时，自动触发 **S 模式定时器中断**（S-Timer interrupt）。

2.3 UART 中断

- 当 UART 外设检测到接收事件或发送状态变化时，会通过 **PLIC (Platform-Level Interrupt Controller)** 向相应的 hart 发出 **S 模式外部中断请求**。
- 进入中断服务程序后，内核会从 **UART 接收 FIFO** 中读取输入缓冲区的数据，并将其内容写入 **UART 发送寄存器 (TX FIFO)** 实现字符回显功能。
- 借助 **S 模式 trap 机制**，操作系统能够在多核系统中对 UART 中断进行统一分发与处理，确保各个 hart 在并发访问串口资源时依然能够稳定、可靠地完成输入输出操作。

3. 实验环境

- 硬件模拟：QEMU RISC-V 64-bit, SMP 2 cores
- 内核代码：自实现 S-mode trap + timer + UART 模块
- 工具链：riscv64-unknown-elf-gcc / riscv64-unknown-elf-ld
- 调试方式：通过 QEMU `-nographic` 终端观察 UART 输出

4. 实验步骤

4.1 处理定时器中断

先处理timer的初始化、创建

```
// 时钟初始化

// called in start.c

void timer_init()
{

    w_menvcfg(r_menvcfg() | (1L<<63)); // 使用SStc extension

    w_mcounteren(r_mcounteren() | 2); //控制S 态能否读一组计数器 CSR 即time
timecmp

    w_stimecmp(r_time() + INTERVAL);

} //借鉴自 xv6-riscv

/*----- 工作在S-mode -----*/

// 系统时钟

static timer_t sys_timer;
```

```

// 时钟创建(初始化系统时钟)

void timer_create()
{
    if(mycpuid()==0)
    {
        sys_timer.ticks = 0;

        spinlock_init(&sys_timer.lk, "timer");
    }

    // 2) 打开 S 态中断:

    // 设置下一个时钟中断时间

    // // - sstatus.SIE: S 态全局中断开关

    w_sstatus(r_sstatus() | SSTATUS_SIE); //允许S态中断
}


// 时钟更新(ticks++ with lock)

//每一次时钟中断都会调用这个函数

void timer_update()
{
    spinlock_acquire(&sys_timer.lk);

    sys_timer.ticks++;

    spinlock_release(&sys_timer.lk);
}

```

```
// 返回系统时钟ticks

uint64 timer_get_ticks()

{

    uint64 t;

    spinlock_acquire(&sys_timer.lk);

    t = sys_timer.ticks;

    spinlock_release(&sys_timer.lk);

    return t;

}
```

然后在 `trap_kernel.c` 里面完成时钟中断处理的函数

```
void timer_interrupt_handler()

{

    if(mycpuid()==0)

    {

        timer_update();

        printf("time %d\n", timer_get_ticks());

    }

    // 设置下一个时钟中断时间
    w_stimecmp(r_time() + INTERVAL);

}
```

4.2处理键盘输入中断

```
// 外设中断处理（基于PLIC）

void external_interrupt_handler()
```

```

{

    int hart = mycpuid();

    int irq = plic_claim();//领取中断号

    switch (irq)
    {

    case 0:                // 无中断

        break;

    case UART_IRQ:        // 串口中断（键盘输入）

        uart_intr();

        break;

    default:

        printf("unexpected PLIC irq=%d on hart=%d\n", irq, hart);

        break;

    }

    if (irq)

        plic_complete(irq);//完成中断处理
}

```

4.3 汇总处理trap的核心逻辑

```

void trap_kernel_handler()

{

    uint64 sepc = r_sepc();                // 记录了发生异常时的pc值

    uint64 sstatus = r_sstatus();          // 与特权模式和中断相关的状态信息

    uint64 scause = r_scause();            // 引发trap的原因
}

```

```

uint64 stval = r_stval();           // 发生trap时保存的附加信息(不同trap不一样)


// 确认trap来自S-mode且此时trap处于关闭状态

assert(sstatus & SSTATUS_SPP, "trap_kernel_handler: not from s-mode");

assert(intr_get() == 0, "trap_kernel_handler: interreput enabled");


int trap_id = scause & 0xf;

int is_interrupt = (scause >> 63) & 1;

// 中断异常处理核心逻辑

if(is_interrupt)
{
    switch (trap_id)
    {
        case 5:

            timer_interrupt_handler();    // 里面会续期: stimecmp = time +
INTERVAL

            return;

        case 9:

            external_interrupt_handler();

            return;

        default:

            break;

    }
}

```

```
}  
  
}
```

4.4 修改main，查看中断是否实现

```
volatile static int started = 0;  
  
int main(void)  
{  
  
    int id = mycpuid();    // 或者用 cpuid()/mycpuid()  
  
    if (id == 0) {  
  
        // 基础初始化  
  
        trap_kernel_init();  
  
        print_init();  
  
        uart_init();  
  
  
        // 安装 S-mode trap 入口（全局一次）  
  
  
        // 本核的中断/PLIC/定时器（每核）  
  
        trap_kernel_inithart();    // 内部应打开 SIE_STIE + SSTATUS_SIE  
  
  
        printf("cpu %d is booting! Sstc timer test starts.\n", id);  
  
  
        __sync_synchronize();  

```

```

started = 1;                // 放行其他核

// 心跳观测循环：每 10 tick 打印一次

uint64 last = timer_get_ticks();

while (1) {

    uint64 t = timer_get_ticks();

    if (t != last) {

        if ((t % 10) == 0) {    // 假设 INTERVAL=0.1s → 约 1 秒打印

            printf("[cpu%d] ticks=%lu\n", id, t);

        }

        last = t;

    }

}

} else {

    // 等待 boot 核完成全局初始化

    while (started == 0) { /* spin */ }

    __sync_synchronize();

    // 每核初始化：打开本核中断 & 预约本核 stimecmp

    trap_kernel_inithart();

```



```
printf("cpu %d is booting! Sstc timer armed.\n", id);

// 次核也跑一个轻量观测（减少刷屏：每 50 tick 打印一次）

uint64 last = timer_get_ticks();

while (1) {

    uint64 t = timer_get_ticks();

    if (t != last) {

        if ((t % 50) == 0) {

            printf("[cpu%d] ticks=%lu\n", id, t);

        }

        last = t;

    }

}

}
```

5 与xv6对比

1. 目前只支持两种trap，少于xv6
2. 代码逻辑更清晰明了

6.实验结果

```
[cpu0] ticks=40
time 41
time 42
time 43
time 44
time 45
time 46
time 47
ctime 48
atime 49
time 50
[cpu0] ticks=50
[cpu1] ticks=50
```

可以看到两个核都能发生时钟中断，且中间键盘输入中端能显示，说明成功

6. 实验总结

- 本次实验成功实现了多核环境下 **S 模式 trap** 的定时器与 **UART 中断管理机制**，系统能够正确响应并处理来自各个外设的中断请求。
- 在实验过程中，深入掌握了 **多核 hart 的启动顺序**、**trap 向量表的配置方法** 以及 **中断触发与响应的完整流程**，进一步理解了操作系统在特权级切换与中断分发中的工作原理。
- 定时器中断由 hart0 负责定期更新时间基（system tick），其余 hart 通过读取该全局计数验证多核定时同步机制的正确性；同时，UART 中断能够即时捕获键盘输入并进行字符回显，充分证明了 **S 模式 trap** 下外设中断处理的稳定性与可靠性。
- 通过本次实验，对 **RISC-V 多核系统中的中断控制与时钟管理机制** 有了更为清晰和具体的理解，并积累了相应的实践经验，为后续的内核调度与设备驱动开发奠定了基础。